Project 2:
Deadline-Driven Scheduler

Leo Barnes – V00981344
Victor Mendes – V00905409
University of Victoria

April 1, 2026

A custom Deadline-Driven Scheduler using FreeRTOS

Table of Contents

Introduction

The objective of this lab was to design and implement a deadline driven scheduler (DDS) using FreeRTOS. The DDS uses the earliest deadline first scheduling algorithm (EDF) to dynamically manage task priorities. The DDS works in conjunction with the existing FreeRTOS task scheduler to correctly prioritize and execute tasks. The tasks are split into two types of tasks, FreeRTOS tasks (F-Tasks) and Deadline-Driven Tasks (DD-Tasks). F-tasks are handled by the FreeRTOS scheduler while DD-tasks are handled by the DDS implementation. Task communication utilizes freeRTOS queues. The DDS can only interact with other tasks through messages delivered through the queues to minimize potential interferences or changes to DDS operations.

Design Solution

The program utilizes a combination of DD-tasks, F-Tasks, timers, queues and a singly linked list implementation to correctly prioritize tasks based on their period. DD-tasks are created using the following struct:

```
typedef struct dd_task {
    TaskHandle_t t_handle;
    task_type type;
    uint32_t task_id;
    uint32_t release_time;
    uint32_t absolute_deadline;
    uint32_t completion_time;
} dd_task;
```

Figure 1:DD task implementation

These task representations are added to a singly linked list that is ordered by earliest absolute deadline first.

Linked list implementation

The singly linked list consists of the following functions implemented in the file dd_list.c and dd_list.h:

```
void add_to_list(dd_task_list **head,dd_task *new_task);
void remove_node(dd_task_list **head, uint32_t task_id);
int find(dd_task_list *head, uint32_t task_id);
```

Figure 2: linked list functions in dd_list.h

Add function

Add_to_list() is a function that adds a new task to a list. The function takes as parameters the head of the list and the new task to be added as a node .The function dynamically allocates memory for the node and places the new node containing the dd_task struct in the list based on it's absolute_deadline value ensuring that the list is always ordered with the earliest deadline first.

```
void add_to_list(dd_task_list **head, dd_task *new_task)
{
    // 1. Allocate new node
        dd_task_list *new_node = pvPortMalloc( xSize: sizeof(dd_task_list));

    if (new_node == NULL) return;

    new_node->task = *new_task;
    new_node->next_task = NULL;

    // 2. Insert at head if list is empty OR earlier deadline
    if (*head == NULL ||
        (*head)->task.absolute_deadline > new_task->absolute_deadline)
    {
        new_node->next_task = *head;
        *head = new_node;
        return;
    }

    // 3. Traverse to find correct insertion point
    dd_task_list *current = *head;

    while (current->next_task != NULL &&
           current->next_task->task.absolute_deadline < new_task->absolute_deadline)
    {
        current = current->next_task;
    }
    // 4. Insert node
    new_node->next_task = current->next_task;
    current->next_task = new_node;
}
```

Figure 3: add_list implementation

Remove function

Remove_node() is the function responsible for removing a task from the list. The function receives the lists head as a parameter and the task id of the task that is to be removed. The function traverses through the list looking for the specified task id. If the task is located, it is removed and its memory is freed. If it is not located or the list is empty, it does nothing.

```
void remove_node(dd_task_list **head, uint32_t task_id) {
    if (head == NULL || *head == NULL) {
        return;
    }
    dd_task_list *current = *head;
    if (current->task.task_id == task_id) {
        *head = current->next_task;
        vPortFree(current);
        return;
    }
    while (current->next_task != NULL &&
           current->next_task->task.task_id != task_id) {
        current = current->next_task;
    }
    if (current->next_task == NULL) {
        printf( format: "Task ID not found\n");
        return;
    }
    dd_task_list *to_remove = current->next_task;
    current->next_task = to_remove->next_task;
    vPortFree(to_remove);
}
```

Figure 4: remove_node implementation

Find Function

find() is the function responsible for locating tasks in a list. The function takes the lists head and a task id as parameters. find() traverses the list checking each tasks id to determine if a specific task exists in the list. If the task is found in the list the function returns 1 otherwise it returns 0.

```
int find(dd_task_list *head, uint32_t task_id){
    dd_task_list *current = head;
        while (current != NULL) {
            if(current->task.task_id == task_id){
                return 1;
            }
            current = current->next_task;
        }
        return 0;
}
```

Figure 5: find implementation

Deadline Driven Scheduler

The entire implementation of the DDS is in main.c.

System overview

The DDS utilizes the following queues:

Name	Purpose
xDDS_Queue	Queue used by tasks to send messages to the DDS
xactive_Queue	Queue used by DDS to pass the active tasks list to the monitor task
xcomplete_Queue	Queue used by DDS to pass the completed tasks list to the monitor task
xoverdue_Queue	Queue used by DDS to pass the overdue tasks list to the monitor task

Table 1: queues used for communication between tasks

Tasks communicate with the DDS by passing messages through freeRTOS queues. Messages contain a message type and task. Message types are mapped to an integer as shown in the image below.

```
typedef struct {
    uint8_t msg_type; // Release, Complete, Get Active list
    dd_task task_info;
} dds_msg;
```

```
#define RELEASE 1
#define COMPLETE 2
#define GET_ACTIVE_LIST 3
#define GET_COMPLETE_LIST 4
#define GET_OVERDUE_LIST 5
```

Figure 6: message struct and message type definitions

The monitor task, the release task and user tasks send messages to the DDS through the DDS queue. The DDS responds to the monitor queue messages by sending the active, completed or overdue lists to its respective queue.

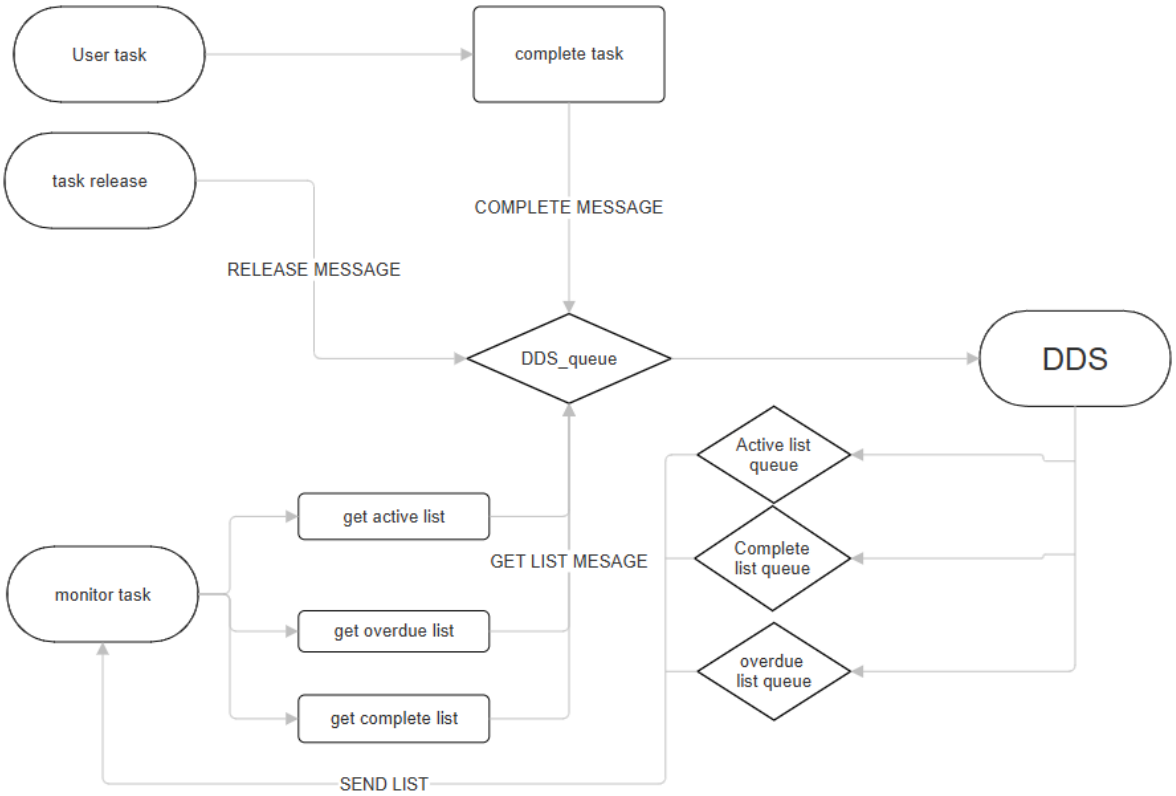

Figure 7: How tasks communicate with the DDS using messages and queues

Tasks releases are controlled by 3 timers each one responsible for one of the three tasks. Each timer value is set to the period of its respective task. When a timer goes off it calls the task generator which sends a task to the DDS. The DDS then sets the correct priority based on EDF and the highest priority task gets executed by the freeRTOS scheduler.

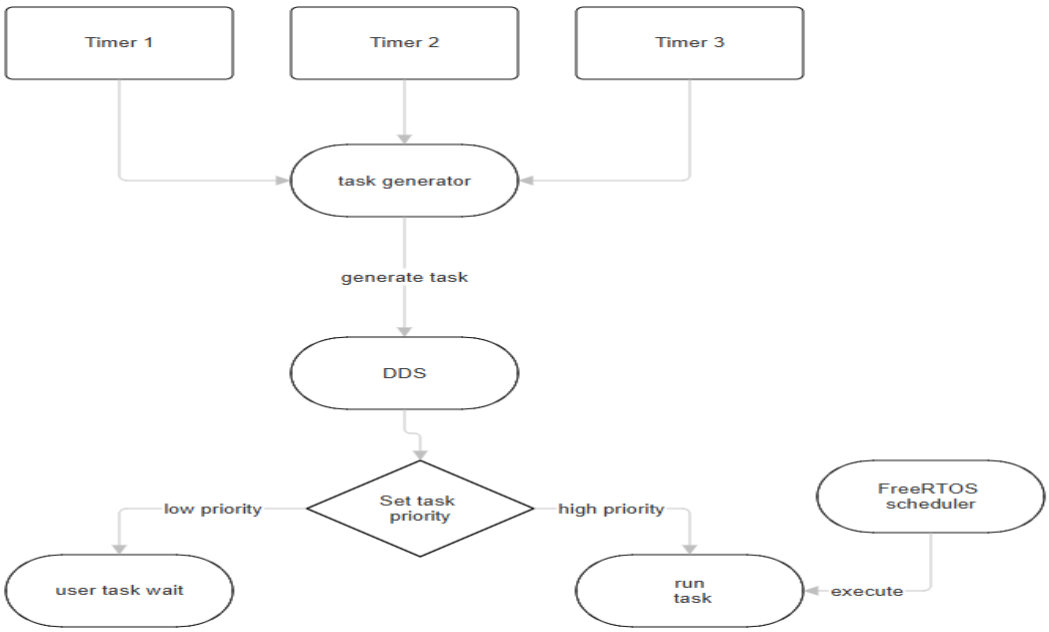

Figure 8: How task release and scheduling occurs

Initialization

Queues and timers are initialized in main at the start of the program. The program then creates all three tasks and releases them since they are all released at t=0. The program then creates the scheduler task and the monitor task using the freeRTOS API. Finaly, the program initiates the freeRTOS scheduler that will work in conjunction with the custom DDS.

```
int main(void)
{
    xDDS_Queue = xQueueCreate(10, sizeof(dds_msg));
    xactive_Queue = xQueueCreate(1, sizeof(dd_task_list*));
    xcomplete_Queue = xQueueCreate(1, sizeof(dd_task_list*));
    xoverdue_Queue = xQueueCreate(1, sizeof(dd_task_list*));

    T1Timer = xTimerCreate( pcTimerName: "T1_Timer", pdMS_TO_TICKS(T1_period), pdTRUE, (void*)1, dd_task_generator);
    T2Timer = xTimerCreate( pcTimerName: "T2_Timer", pdMS_TO_TICKS(T2_period), pdTRUE, (void*)2, dd_task_generator);
    T3Timer = xTimerCreate( pcTimerName: "T3_Timer", pdMS_TO_TICKS(T3_period), pdTRUE, (void*)3, dd_task_generator);

    xTimerStart(T1Timer, 0);
    xTimerStart(T2Timer, 0);
    xTimerStart(T3Timer, 0);

    init_task();

    printf( format: "Main\n");
    xTaskCreate(dd_scheduler, pcName: "DDS", configMINIMAL_STACK_SIZE, NULL, uxPriority: dds_PRIORITY, NULL);
    xTaskCreate(monitor_task, pcName: "Monitor", configMINIMAL_STACK_SIZE, NULL, uxPriority: monitor_PRIORITY, NULL);

    vTaskStartScheduler();
   for(;;);
}
```

Figure 9: main portion of the code initializing timers, queues ,tasks and scheduler

Tasks

The DDS consists of F tasks and helper functions. F-tasks are tasks that use the freeRTOS task creation and scheduling implementation. Task priorities are based on the configMAX_Priorities constant which is set to 7. The DDS has the highest priority; the monitor has the second highest priority followed by the task generator as the third highest. User tasks can either be set to low priority or high priority as determined by the DDS.

```
#define dds_PRIORITY          (configMAX_PRIORITIES - 1)
#define monitor_PRIORITY      (configMAX_PRIORITIES - 2)
#define generator_PRIORITY    (configMAX_PRIORITIES - 3)
#define user_task_LOW         (1)
#define user_task_HIGH        (configMAX_PRIORITIES - 4)
```

Figure 10: priority configuration of F-tasks

Deadline driven scheduler task

The dd_scheduler function is responsible for scheduling task priority. Dd_scheduler creates 3 lists of tasks. The active_list contains all the tasks that are currently active. The completed_list contains all the tasks that have been completed. The overdue_list contains all the tasks that failed to complete by its deadline.

The dd_scheduler checks the xDDS_queue for a new message. When a new message arrives, it contains one of 5 message types. The Dd_scheduler Contains a switch statement corresponding to each message type.

If the message type is “RELEASE” the dd_scheduler has received a new task by the task releaser. The scheduler checks if that task is present in active task list. If it is present, it means that the task has not completed in time and that task is removed from the active list and is added to the overdue list. Following the check for overdue tasks the new task with the new absolute deadline is added to the active task list.

If the message type is “COMPLETE” a user task has completed. That task is removed from the active task list and is added to the completed task list.

If the message type is “GET_ACTIVE_LIST”, “GET_COMPLETE_LIST” OR “GET_OVERDUE_LIST” then the scheduler sends the requested list to xactive_Queue, xcomplete_Queue or xoverdue_Queu depending on which list is being requested.

```c
/* --- Core Scheduler Logic --- */
static void dd_scheduler(void *pvParameters) {
    dd_task_list* active_list_head = NULL;
    dd_task_list* completed_list_head = NULL;
    dd_task_list* overdue_list_head = NULL;
    dds_msg rcvd_msg;
    for(;;) {
        if(xQueueReceive(xDDS_Queue, &rcvd_msg, portMAX_DELAY)) {
            uint32_t current_time = xTaskGetTickCount();
            switch (rcvd_msg.msg_type)
            {
                case RELEASE:
                    if(find(active_list_head, rcvd_msg.task_info.task_id)){
                        remove_node(&active_list_head, rcvd_msg.task_info.task_id);
                        add_to_list(&overdue_list_head, &rcvd_msg.task_info);
                    }
                    add_to_list(&active_list_head, &rcvd_msg.task_info);
                    printf( format: "Task: %u, Release: %u, Deadline %u\n", rcvd_msg.task_info.task_id , rcvd_msg.task_info.release_time, rcvd_msg.task_info.absolute_deadline);
                    break;
                case COMPLETE:
                    rcvd_msg.task_info.completion_time = current_time;
                    remove_node(&active_list_head, rcvd_msg.task_info.task_id);
                    add_to_list(&completed_list_head, &rcvd_msg.task_info);
                    printf( format: "Task: %u, Complete %u\n", rcvd_msg.task_info.task_id, rcvd_msg.task_info.completion_time);
                    break;
                case GET_ACTIVE_LIST:
                    xQueueSend(xactive_Queue, &active_list_head, 0);
                    break;
                case GET_COMPLETE_LIST:
                    xQueueSend(xcomplete_Queue, &completed_list_head, 0);
                    break;
                case GET_OVERDUE_LIST:
                    xQueueSend(xoverdue_Queue, &overdue_list_head, 0);
                    break;
                default:
                    break;
            }
            // --- EDF PRIORITY SWAP ---
            assign_task_priorities(active_list_head);
        }
    }
}
```

Figure 11: dd_scheduler implementation

After dealing with the message the scheduler traverses the active task list setting the head of the list to high priority and every other task to low priority by calling assign_task_priorities().

```c
void assign_task_priorities(dd_task_list *head){
    dd_task_list *current = head;
    if (current == NULL) {
        return; // no tasks to schedule
    }
    // 1. First task in the list gets HIGH priority
    vTaskPrioritySet(current->task.t_handle, uxNewPriority: user_task_HIGH);
    // 2. All remaining tasks get LOW priority
    current = current->next_task;
    while (current != NULL) {
        vTaskPrioritySet(current->task.t_handle, uxNewPriority: user_task_LOW);
        current = current->next_task;
    }
}
```

Figure 12:Function responsible for setting task priorities in list

Monitor task

The monitor task is responsible for printing information about completed, overdue and active tasks every hyper period. The monitor task communicates with the dd_scheduler through three different functions each one responsible for requesting and obtaining either the active task list, the overdue task list or the complete task list. The functions send messages to the DDS_Queue and then wait for the dd_scheduler to send the list back through their respective queue and finally returns the list to the monitor task.

```
dd_task_list* get_active_dd_task_list(void) {
    dds_msg msg;
    msg.msg_type = GET_ACTIVE_LIST;
    dd_task_list* active_list_head = NULL;
    xQueueSend(xDDS_Queue, &msg, 0);
    xQueueReceive(xactive_Queue, &active_list_head, portMAX_DELAY);
    return active_list_head;
}

dd_task_list* get_complete_dd_task_list(void) {
    dds_msg msg;
    msg.msg_type = GET_COMPLETE_LIST;
    dd_task_list* complete_list_head = NULL;
    xQueueSend(xDDS_Queue, &msg, 0);
    xQueueReceive(xcomplete_Queue, &complete_list_head, portMAX_DELAY);
    return complete_list_head;
}

dd_task_list* get_overdue_dd_task_list(void) {
    dds_msg msg;
    msg.msg_type = GET_OVERDUE_LIST;
    dd_task_list* overdue_list_head = NULL;
    xQueueSend(xDDS_Queue, &msg, 0);
    xQueueReceive(xoverdue_Queue, &overdue_list_head, portMAX_DELAY);
    return overdue_list_head;
}
```

Figure 13:Functions that request and receive task lists

The monitor task then counts how many nodes exist in each other lists and prints those numbers to the console.

```
static void monitor_task(void *pvParameters) {
    for(;;) {
        vTaskDelay(pdMS_TO_TICKS(HyperPeriod));

        dd_task_list *active = get_active_dd_task_list();
        dd_task_list *complete = get_complete_dd_task_list();
        dd_task_list *overdue = get_overdue_dd_task_list();

        uint32_t active_count = 0, complete_count = 0, overdue_count = 0;

        dd_task_list *curr = active;
        while(curr != NULL) { active_count++; curr = curr->next_task; }

        curr = complete;
        while(curr != NULL) { complete_count++; curr = curr->next_task; }

        curr = overdue;
        while(curr != NULL) { overdue_count++; curr = curr->next_task; }

        printf( format: "\n--- Monitor Report (%u ms) ---\n", (unsigned int)xTaskGetTickCount());
        printf( format: "Active Tasks: %u\n", (unsigned int)active_count);
        printf( format: "Completed Tasks: %u\n", (unsigned int)complete_count);
        printf( format: "Overdue Tasks: %u\n", (unsigned int)overdue_count);
    }
}
```

Figure 14:monitor task implementation

User tasks

All three user tasks use one single implementation. The task uses its ID to determine its period and then loops using the condition (current time – start time < execution time). When the condition fails it means that has concluded its execution time. The user task then calls complete_dd_task() passing its task id and suspends itself.

```
static void user_defined_task(void *pvParameters) {
    uint32_t my_id = (uint32_t)pvParameters;

    uint32_t execution_time_ms;

    if (my_id == 1) execution_time_ms = T1_exTime;
    else if (my_id == 2) execution_time_ms = T2_exTime;
    else if (my_id == 3) execution_time_ms = T3_exTime;

    for(;;) {

        uint32_t start_tick = xTaskGetTickCount();
        while ((xTaskGetTickCount() - start_tick) < pdMS_TO_TICKS(execution_time_ms)) { }

        complete_dd_task(my_id);

        vTaskSuspend(NULL);
    }
}
```

Figure 15: user defined task behaviour

Complete_dd_task() creates a message of type “COMPLETE” and sends it to xDDS_Queue resulting in the task being removed and deprioritized by the dd scheduler.

```
void complete_dd_task(uint32_t task_id) {
    dds_msg msg;
    msg.msg_type = COMPLETE;
    msg.task_info.task_id = task_id;

    xQueueSend(xDDS_Queue, &msg, 0);
}
```

Figure 16: complete_dd_task implementation

Task generation and task initialization

The dd_task_generator is responsible for releasing tasks except for at time 0 which is controlled by the task initializer that is called once in the main portion of the code. The task initializer creates the three tasks and informs the dd scheduler.

```
void init_task(){
    xTaskCreate(user_defined_task, pcName:"T1", configMINIMAL_STACK_SIZE, (void*)1, uxPriority:user_task_LOW, &xTask1);
    release_dd_task(xTask1, type:PERIODIC, task_id:1, release:0, deadline:T1_period);
    xTaskCreate(user_defined_task, pcName:"T2", configMINIMAL_STACK_SIZE, (void*)2, uxPriority:user_task_LOW, &xTask2);
    release_dd_task(xTask2, type:PERIODIC, task_id:2, release:0, deadline:T2_period);
    xTaskCreate(user_defined_task, pcName:"T3", configMINIMAL_STACK_SIZE, (void*)3, uxPriority:user_task_LOW, &xTask3);
    release_dd_task(xTask3, type:PERIODIC, task_id:3, release:0, deadline:T3_period);
}
```

Figure 17:Task initialization at t=0

The dd_task_generator () function is called when either timer 1, timer 2 or timer 3 completes their countdown. The function checks which timer called it and if the task is set to periodic to determine which task to resume and release to the dd scheduler

```
static void dd_task_generator(TimerHandle_t xTimer) {
    uint32_t timerID = (uint32_t) pvTimerGetTimerID(xTimer);
    uint32_t now = xTaskGetTickCount();

    if(timerID == 1 && T1_type == PERIODIC){
        vTaskResume(xTask1);
        release_dd_task(xTask1, type:T1_type, task_id:1, release:now, deadline:T1_period);

    }else if(timerID == 2&& T2_type == PERIODIC){
        vTaskResume(xTask2);
        release_dd_task(xTask2, type:T2_type, task_id:2, release:now, deadline:T2_period);

    }else if(timerID == 3 && T3_type == PERIODIC){
        vTaskResume(xTask3);
        release_dd_task(xTask3, type:T3_type, task_id:3, release:now, deadline:T3_period);
    }
}
```

Figure 18:task generation implementation

Release_dd_task() creates a task struct using task parameters passed to it as arguments and creates a new message of type “RELEASE.” Release_dd_task sends the “RELEASE” type message containing the task struct to xDDS_queue where it will be picked up by the dd scheduler, added to the active tasks list and given the appropriate priority.

```
/* --- Interface Implementations --- */
void release_dd_task(TaskHandle_t t_handle, task_type type, uint32_t task_id, uint32_t release, uint32_t deadline) {
    dds_msg msg;
    msg.msg_type = RELEASE;
    msg.task_info.t_handle = t_handle;
    msg.task_info.type = type;
    msg.task_info.task_id = task_id;
    msg.task_info.release_time = release;
    msg.task_info.absolute_deadline = release + deadline;
    xQueueSend(xDDS_Queue, &msg, 0);
}
```

Figure 19: Release dd task function

Discussion

The following section discusses how the scheduling, architecture, list management and task communication was successfully implemented for the DDS.

Scheduling

The core of the system relies on dynamic priority manipulation. Through the use of an *assign_task_priorities* function the system ensures that the task at the head of the active list is always granted a high priority (*user_task_HIGH*) while all other task are granted a lower priority (*user_task_LOW*). As the active list is sorted by absolute deadline this results with the FreeRTOS schedules always executing the task with the earliest deadline. Any task that aren’t completed by their deadline are immediately stopped and moved to a overdue list so the scheduler can prioritize other tasks.

Task Communication

The system maintains a strict separation between the DDS and auxiliary task. All communication is performed using FreeRTOS queues such as, *xDDS_Queue* or *xactive_Queue.* Task are generated using a *dd_task_generator* function which uses timers to trigger the release of tasks. When a timer expires a *release_dd_task* function packages the data and send its to the DDS via a queue. Once the task is completed another task is then used to signal the DDS to move the task from the active queue to the completed queue. Task are created and deleted frequently so the system relied on *heap_4.c* to ensure that memory was managed effectively. This delete/create method allows for simpler tasks that run independently rather than a universal task that gets reused and must be reset each call.

List Management

A custom linked-list structure is used to manage the task. This allows more freedom compared to the built-in lists functions FreeRTOS provides. With the custom list structure, the system can automatically organize the list by deadline whenever a new task is added. Additionally, the DDS can perform checks during the release case to find missed deadlines. If a task ID is already in the active list when a new release occurs it indicates a deadline has been missed and is moved to the overdue list.

Results and Analysis

The DDS system was evaluated using 3 test benches *(Table 4)* to verify its ability to handle different processor loads and task periods. Test Bench 1 was nearly successful, while all tasks ran at the correct starting times as seen in *Figure 20,* there was issues with the completion times. The first tasks that run match closely to the expect values with results 97ms, 247ms, and 497. These first results while close to the expected were slightly displaced most likely because of the print statements adding overhead to the system. Further along though there is an issue with task 2 ending early as seen in *Table 2*, this could be caused by it running in parallel to task 1 and therefor ending 150 ms after it started rather than the intended 150 + 95 ms.

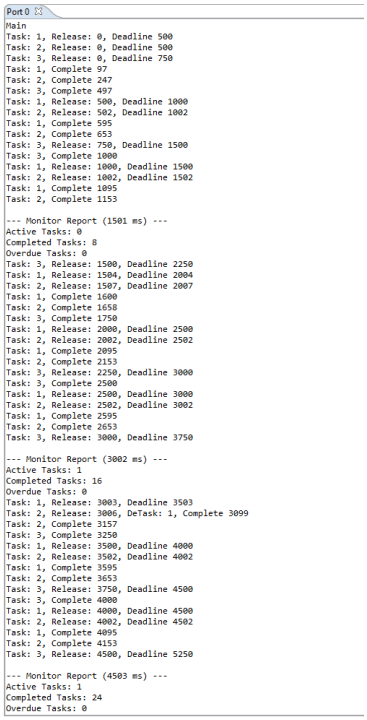

Figure 20. Test bench 1 results.

Event #	*Event*	*Measured Time*	*Expected Time*
1	*Task 1 Released*	*0*	*0*
2	*Task 2 Released*	*0*	*0*
3	*Task 3 Released*	*0*	*0*
4	*Task 1 Compete*	*97*	*95*
5	*Task 2 Compete*	*247*	*245*
6	*Task 3 Compete*	*497*	*495*
7	*Task 1 Released*	*500*	*500*
8	*Task 2 Released*	*500*	*500*
9	*Task 1 Compete*	*595*	*595*
10	*Task 2 Compete*	*653*	*745*
11	*Task 3 Release*	*750*	*750*
12	*Task 3 Compete*	*1000*	*1000*
13	*Task 1 Released*	*1000*	*1000*
14	*Task 2 Released*	*1002*	*1000*
15	*Task 1 Compete*	*1095*	*1095*
16	*Task 2 Compete*	*1053*	*1245*
17	*Task 1 Release*	*1504*	*1500*

Table 2. DDS events for Test Bench #1

Continuing to test bench 2 the same errors occurred as in test bench 1. As seen in *Figure 21 and Table 3*, the monitor results are the same despite the errors that occur with task 2. This highlights the importance of checking the results multiple ways rather than just a single task monitor.

	Measured	Expected
Number of Active DD tasks	*0*	*0*
Number of Completed DD tasks	*11*	*11*
Number of Overdue DD tasks	*0*	*0*

Table 3. Monitor Output for Test #2 after 1 hyper-period

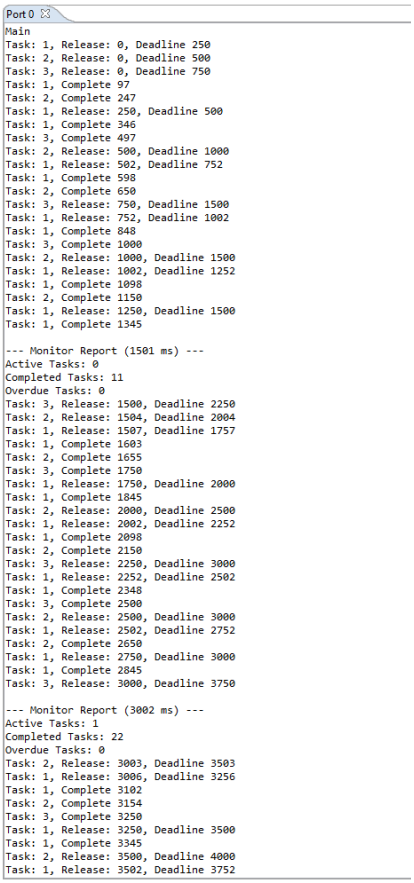

Figure 21. Test bench 2 results

As for Test Bench 3, this is where the overdue tasks are tested. The system results for this task showed the same task 2 error but this time introduced a new bug surrounding the timers. In this new test all tasks are set to the same period so when all timer releases at the same time it causes the tasks start times to be staggered slightly. This staggered task start time mixed with the overhead caused overdue tasks but our system was successful at catching those tasks and moving them to the overdue list. The results confirmed that the custom scheduler, while having some issues, was successful at providing a functional EDF framework. Despite the observed timer issues and staggered release times the scheduler was able to demonstrate error handling by correctly moving overdue tasks to the overdue list. With a little more time, this project could easily be debugged and working smoothly.

	Test Bench #1		*Test Bench #2*		*Test Bench #3*	
Task	*Execution Time (ms)*	*Period (ms)*	*Execution Time (ms)*	*Period (ms)*	*Execution Time (ms)*	*Period*
1	*95*	*500*	*95*	*250*	*100*	*500*
2	*150*	*500*	*150*	*500*	*200*	*500*
3	*250*	*750*	*250*	*750*	*200*	*500*

Table 4. Test bench 2 results

Limitations and Possible Improvements

Overall, the lab is a good entry point for exploring how a EDF scheduler works in real time computer systems.

Limitations

Currently a major limitation is the use of *printf* statements. Every call to a *printf* causes significant overhead and timing. In a hard real-time system this could interfere with the task execution time and potentially cause avoidable deadline misses. While important for testing/confirmation in this project, once working all print statements could be easily removed for a more efficient system. Additionally the *user_defined_task* uses a while loop to simulate execution time. This may be effective for testing, but it prevents the CPU from entering low-power states or allowing lower-priority F-tasks to run effectively. Lastly, the system requires hard-coded values for both period and execution time, as well as timers for each task, this makes the systems scalability minimal. For each new task we add we must modify the code significantly to add the new parameters and the timers. This project was effective for showcasing the internal cogs of a scheduler but would not work as a larger scale scheduler.

Improvements

Some Improvements that could be made to the system include, an overdue watchdog, memory optimization, and CPU utilization reporting. To further ensure that overdue tasks are handled effectively a watchdog timer could be started for each DD-task and set to the task's absolute deadline. If the timer expires before *complete_dd_task* is called, a callback could immediately move the task to the overdue list. Additionally, as the manual suggests the monitor task could be improved to calculate the actual CPU utilization. This would involve tracking the total execution time of a task versus the total elapsed time. Finally, the current design uses a complete task list which consumes valuable memory. As task are created and completed, they continuously fill the completed and overdue lists and take more and more memory as they are never deleted or removed. To get around this an *int* could be used to track the completed and overdue task count, and a shorter list could be used to store the most recent tasks, clearing old ones out. The timer staggering issue could be fixed by using one single timer set to the greatest common denominator of the task periods instead of 3 separate timers. Instead of having 3 timers attempting to execute at the same time a single timer could release all the tasks since the single timer would activate at every possible release time.

Summary

This Project successfully demonstrated a custom deadline-driven scheduler implemented as a high priority FreeRTOS task. By using EDF algorithm, the system actively managed tasks by organizing them into active, complete, and overdue linked lists based on their deadlines. The core mechanics of the system relied on dynamic priority updating where the task with the nearest deadline was granted high priority while all other were set to low. All communication between task was handled by queues and timers.

The project was verified using a monitor task that reported the system statistics, such as the count of overdue and active tasks. While the system was able to effectively run the required test benches, there were a few limitations such as large overhead and limited scalability. Future improvements could include a watchdog timer for overdue tasks, and better memory optimization. Overall, the project provided great experience in understanding how schedulers work in a priority abased real-time system.

Appendix

The following is the entire code implementation.

dd_list.c

```
#include "dd_list.h"
#include <stdio.h>
#include <stdlib.h>


void add_to_list(dd_task_list **head, dd_task *new_task)
{
   // 1. Allocate new node
      dd_task_list *new_node = pvPortMalloc(sizeof(dd_task_list));

   if (new_node == NULL) return;

   new_node->task = *new_task;
   new_node->next_task = NULL;

   // 2. Insert at head if list is empty OR earlier deadline
   if (*head == NULL ||
      (*head)->task.absolute_deadline > new_task->absolute_deadline)
   {
      new_node->next_task = *head;
      *head = new_node;
      return;
   }

   // 3. Traverse to find correct insertion point
   dd_task_list *current = *head;

   while (current->next_task != NULL &&
         current->next_task->task.absolute_deadline < new_task->absolute_deadline)
```

```
    {
        current = current->next_task;
    }
    // 4. Insert node
    new_node->next_task = current->next_task;
    current->next_task = new_node;
}


void remove_node(dd_task_list **head, uint32_t task_id) {
    if (head == NULL || *head == NULL) {
        return;
    }
    dd_task_list *current = *head;
    if (current->task.task_id == task_id) {
        *head = current->next_task;
        vPortFree(current);
        return;
    }
    while (current->next  task != NULL &&
           current->next  task->task.task  id != task  id) {
        current = current->next_task;
    }
    if (current->next_task == NULL) {
        printf("Task ID not found\n");
        return;
    }
    dd_task_list *to_remove = current->next_task;
    current->next_task = to_remove->next_task;
    vPortFree(to_remove);
}


int find(dd_task_list *head, uint32_t task_id){
     dd_task_list *current = head;
        while (current != NULL) {
          if(current->task.task_id == task_id){
            return 1;
          }
           current = current->next  task;
        }
        return 0;
}
```

main.c

```
/* Standard includes. */
#include <stdint.h>
#include <stdio.h>
#include "stm32f4_discovery.h"
/* Kernel includes. */
#include "stm32f4xx.h"
#include "../FreeRTOS_Source/include/FreeRTOS.h"
#include "../FreeRTOS_Source/include/queue.h"
#include "../FreeRTOS  Source/include/semphr.h"
#include "../FreeRTOS_Source/include/task.h"
#include "../FreeRTOS  Source/include/timers.h"
#include "dd  list.h"


/*-----------------------------------------------------------*/
#define dds  PRIORITY          (configMAX  PRIORITIES - 1)
#define monitor_PRIORITY       (configMAX_PRIORITIES - 2)
```

```
#define generator_PRIORITY    (configMAX_PRIORITIES - 3)
#define user_task_LOW         (1)
#define user_task_HIGH        (configMAX_PRIORITIES - 4)

#define RELEASE 1
#define COMPLETE 2
#define GET  ACTIVE  LIST 3
#define GET_COMPLETE_LIST 4
#define GET  OVERDUE  LIST 5



/********************TEST BENCH 1*********************/

//#define T1  period 500
//#define T2_period 500
//#define T3_period 750
//
//#define T1_exTime 95
//#define T2  exTime 150
//#define T3_exTime 250
//
//#define T1_type PERIODIC
//#define T2_type PERIODIC
//#define T3_type PERIODIC
//
//#define HyperPeriod 1500


/* ******************* TEST BENCH 2*********************/

 #define T1_period 250
 #define T2  period 500
 #define T3_period 750

 #define T1  exTime 95
 #define T2_exTime 150
 #define T3  exTime 250

#define T1_type PERIODIC
#define T2_type PERIODIC
#define T3_type PERIODIC

#define HyperPeriod 1500

/******************* TEST BENCH 3***********************/
//
// #define T1_period 500
// #define T2  period 500
// #define T3_period 500
//
// #define T1_exTime 100
// #define T2_exTime 200
// #define T3_exTime 200
//
//#define T1_type PERIODIC
//#define T2_type PERIODIC
//#define T3_type PERIODIC
//
//#define HyperPeriod 1500

/* ************************************************** */


typedef struct {
```

```
    uint8_t msg_type; // Release, Complete, Get Active list
    dd_task task_info;
} dds_msg;

QueueHandle_t xDDS_Queue;
QueueHandle_t xactive_Queue;
QueueHandle_t xcomplete_Queue;
QueueHandle_t xoverdue_Queue;

static void dd_scheduler(void *pvParameters);
static void monitor_task(void *pvParameters);

static void user_defined_task(void *pvParameters);

TimerHandle_t T1Timer;
TimerHandle_t T2Timer;
TimerHandle_t T3Timer;

TaskHandle_t xTask1, xTask2, xTask3;

static void assign_task_priorities(dd_task_list *head);


void release_dd_task(TaskHandle_t t_handle, task_type type, uint32_t task_id, uint32_t release, uint32_t deadline);
void complete_dd_task(uint32_t task_id);
static void dd_task_generator();
static void init_task();

dd_task_list *get_active_dd_task_list(void);
dd_task_list *get_complete_dd_task_list(void);
dd_task_list *get_overdue_dd_task_list(void);


/*-----------------------------------------------------------*/

int main(void)
{
    xDDS_Queue = xQueueCreate(10, sizeof(dds_msg));
    xactive_Queue = xQueueCreate(1, sizeof(dd_task_list*));
    xcomplete_Queue = xQueueCreate(1, sizeof(dd_task_list*));
    xoverdue_Queue = xQueueCreate(1, sizeof(dd_task_list*));

    T1Timer = xTimerCreate("T1_Timer", pdMS_TO_TICKS(T1_period), pdTRUE, (void*)1, dd_task_generator);
    T2Timer = xTimerCreate("T2_Timer", pdMS_TO_TICKS(T2_period), pdTRUE, (void*)2, dd_task_generator);
    T3Timer = xTimerCreate("T3_Timer", pdMS_TO_TICKS(T3_period), pdTRUE, (void*)3, dd_task_generator);
    xTimerStart(T1Timer, 0);
    xTimerStart(T2Timer, 0);
    xTimerStart(T3Timer, 0);

    init_task();

    printf("Main\n");
    xTaskCreate(dd_scheduler, "DDS", configMINIMAL_STACK_SIZE, NULL, dds_PRIORITY, NULL);
    xTaskCreate(monitor_task, "Monitor", configMINIMAL_STACK_SIZE, NULL, monitor_PRIORITY, NULL);

    vTaskStartScheduler();
   for(;;);
}


/* --- Core Scheduler Logic --- */
static void dd_scheduler(void *pvParameters) {
    dd_task_list* active_list_head = NULL;
    dd_task_list* completed_list_head = NULL;
    dd_task_list* overdue_list_head = NULL;
    dds_msg rcvd_msg;
```

```
    for(;;) {
        if(xQueueReceive(xDDS_Queue, &rcvd_msg, portMAX_DELAY)) {
            uint32_t current_time = xTaskGetTickCount();
            switch (rcvd_msg.msg_type)
            {
                case RELEASE:
                    if(find(active list head, rcvd msg.task info.task id)){
                        remove_node(&active_list_head, rcvd_msg.task_info.task_id);
                        add to list(&overdue list head, &rcvd msg.task info);
                    }
                    add_to_list(&active_list_head, &rcvd_msg.task_info);
                    printf("Task: %u, Release: %u, Deadline %u\n", rcvd msg.task info.task id , rcvd msg.task info.release time,
rcvd_msg.task_info.absolute_deadline);
                    break;
                case COMPLETE:
                    rcvd_msg.task_info.completion_time = current_time;
                    remove_node(&active_list_head, rcvd_msg.task_info.task_id);
                    add_to_list(&completed_list_head, &rcvd_msg.task_info);
                    printf("Task: %u, Complete %u\n", rcvd_msg.task_info.task_id, rcvd_msg.task_info.completion_time);
                    break;
                case GET ACTIVE LIST:
                    xQueueSend(xactive_Queue, &active_list_head, 0);
                    break;
                case GET_COMPLETE_LIST:
                    xQueueSend(xcomplete Queue, &completed list head, 0);
                    break;
                case GET OVERDUE LIST:
                    xQueueSend(xoverdue_Queue, &overdue_list_head, 0);
                    break;
                default:
                    break;
            }
            // --- EDF PRIORITY SWAP ---
            assign_task_priorities(active_list_head);
        }
    }
}

/* --- Interface Implementations --- */
void release dd task(TaskHandle t t handle, task type type, uint32 t task id, uint32 t release, uint32 t deadline) {
    dds_msg msg;
    msg.msg_type = RELEASE;
    msg.task info.t handle = t handle;
    msg.task_info.type = type;
    msg.task_info.task_id = task_id;
    msg.task_info.release_time = release;
    msg.task_info.absolute_deadline = release + deadline;
    xQueueSend(xDDS_Queue, &msg, 0);
}

void complete_dd_task(uint32_t task_id) {
    dds msg msg;
    msg.msg_type = COMPLETE;
    msg.task info.task id = task id;

    xQueueSend(xDDS Queue, &msg, 0);
}

dd_task_list* get_active_dd_task_list(void) {
    dds_msg msg;
    msg.msg_type = GET_ACTIVE_LIST;
    dd task list* active list head = NULL;
    xQueueSend(xDDS_Queue, &msg, 0);
    xQueueReceive(xactive_Queue, &active_list_head, portMAX_DELAY);
    return active list head;
}
```

```
dd_task_list* get_complete_dd_task_list(void) {
    dds_msg msg;
    msg.msg_type = GET_COMPLETE_LIST;
    dd_task_list* complete_list_head = NULL;
    xQueueSend(xDDS_Queue, &msg, 0);
    xQueueReceive(xcomplete  Queue, &complete  list  head, portMAX  DELAY);
    return complete_list_head;
}

dd_task_list* get_overdue_dd_task_list(void) {
    dds  msg msg;
    msg.msg_type = GET_OVERDUE_LIST;
    dd_task_list* overdue_list_head = NULL;
    xQueueSend(xDDS_Queue, &msg, 0);
    xQueueReceive(xoverdue_Queue, &overdue_list_head, portMAX_DELAY);
    return overdue_list_head;
}

static void dd  task  generator(TimerHandle  t xTimer) {
    uint32  t timerID = (uint32  t) pvTimerGetTimerID(xTimer);
    uint32_t now = xTaskGetTickCount();

    if(timerID == 1 && T1_type == PERIODIC){
      vTaskResume(xTask1);
      release_dd_task(xTask1, T1_type, 1, now, T1_period);

    }else if(timerID == 2&& T2_type == PERIODIC){
      vTaskResume(xTask2);
      release_dd_task(xTask2, T2_type, 2, now, T2_period);

    }else if(timerID == 3 && T3_type == PERIODIC){
      vTaskResume(xTask3);
      release_dd_task(xTask3, T3_type, 3, now, T3_period);
    }
}

void init  task(){
    xTaskCreate(user_defined_task, "T1", configMINIMAL_STACK_SIZE, (void*)1, user_task_LOW, &xTask1);
    release  dd  task(xTask1, T1  type, 1, 0, T1  period);
    xTaskCreate(user_defined_task, "T2", configMINIMAL_STACK_SIZE, (void*)2, user_task_LOW, &xTask2);
    release_dd_task(xTask2, T2_type, 2, 0, T2_period);
    xTaskCreate(user  defined  task, "T3", configMINIMAL  STACK  SIZE, (void*)3, user  task  LOW, &xTask3);
    release_dd_task(xTask3, T3_type, 3, 0, T3_period);
}

static void user_defined_task(void *pvParameters) {
    uint32_t my_id = (uint32_t)pvParameters;

    uint32_t execution_time_ms;

    if (my  id == 1) execution  time  ms = T1  exTime;
    else if (my_id == 2) execution_time_ms = T2_exTime;
    else if (my  id == 3) execution  time  ms = T3  exTime;

    for(;;) {

        uint32_t start_tick = xTaskGetTickCount();
        while ((xTaskGetTickCount() - start_tick) < pdMS_TO_TICKS(execution_time_ms)) { }

        complete_dd_task(my_id);

        vTaskSuspend(NULL);
    }
}
```

```
static void monitor_task(void *pvParameters) {
    for(;;) {
        vTaskDelay(pdMS_TO_TICKS(HyperPeriod));

        dd_task_list *active = get_active_dd_task_list();
        dd_task_list *complete = get_complete_dd_task_list();
        dd_task_list *overdue = get_overdue_dd_task_list();

        uint32_t active_count = 0, complete_count = 0, overdue_count = 0;

        dd_task_list *curr = active;
        while(curr != NULL) { active_count++; curr = curr->next_task; }

        curr = complete;
        while(curr != NULL) { complete_count++; curr = curr->next_task; }

        curr = overdue;
        while(curr != NULL) { overdue_count++; curr = curr->next_task; }

        printf("\n--- Monitor Report (%u ms) ---\n", (unsigned int)xTaskGetTickCount());
        printf("Active Tasks: %u\n", (unsigned int)active_count);
        printf("Completed Tasks: %u\n", (unsigned int)complete_count);
        printf("Overdue Tasks: %u\n", (unsigned int)overdue_count);
    }
}

void assign_task_priorities(dd_task_list *head){
    dd_task_list *current = head;
    if (current == NULL) {
        return; // no tasks to schedule
    }
    // 1. First task in the list gets HIGH priority
    vTaskPrioritySet(current->task.t_handle, user_task_HIGH);
    // 2. All remaining tasks get LOW priority
    current = current->next_task;
    while (current != NULL) {
        vTaskPrioritySet(current->task.t_handle, user_task_LOW);
        current = current->next_task;
    }
}

void vApplicationMallocFailedHook( void )
{
    /* The malloc failed hook is enabled by setting
    configUSE_MALLOC_FAILED_HOOK to 1 in FreeRTOSConfig.h.

    Called if a call to pvPortMalloc() fails because there is insufficient
    free memory available in the FreeRTOS heap.  pvPortMalloc() is called
    internally by FreeRTOS API functions that create tasks, queues, software
    timers, and semaphores.  The size of the FreeRTOS heap is set by the
    configTOTAL_HEAP_SIZE configuration constant in FreeRTOSConfig.h. */
    for( ;; );
}
/*-----------------------------------------------------------*/

void vApplicationStackOverflowHook( xTaskHandle pxTask, signed char *pcTaskName )
{
    ( void ) pcTaskName;
    ( void ) pxTask;

    /* Run time stack overflow checking is performed if
    configconfigCHECK_FOR_STACK_OVERFLOW is defined to 1 or 2.  This hook
    function is called if a stack overflow is detected.  pxCurrentTCB can be
    inspected in the debugger if the task name passed into this function is
    corrupt. */
    for( ;; );
```

```
}
/*-----------------------------------------------------------*/

void vApplicationIdleHook( void )
{
volatile size_t xFreeStackSpace;

    /* The idle task hook is enabled by setting configUSE_IDLE_HOOK to 1 in
    FreeRTOSConfig.h.

    This function is called on each cycle of the idle task.  In this case it
    does nothing useful, other than report the amount of FreeRTOS heap that
    remains unallocated. */
    xFreeStackSpace = xPortGetFreeHeapSize();

    if( xFreeStackSpace > 100 )
    {
      /* By now, the kernel has allocated everything it is going to, so
      if there is a lot of heap remaining unallocated then
      the value of configTOTAL  HEAP  SIZE in FreeRTOSConfig.h can be
      reduced accordingly. */
    }
}
```